M14 - Kafka + Streaming Foundations

Zero-to-Job-Ready Data Engineer 2026 | RC1 | 12 beginner-complete lessons

BEGINNER SUFFICIENCY RULE
This book must make Kafka understandable before commands appear. The goal is event-streaming reasoning: identity, partitioning, ordering, offsets, consumer groups, replay, delivery, schemas, lag and recovery.

PINNED BASELINE
Kafka 4.3.1 + Java 17+, confluent-kafka 2.15.0, and PySpark 4.2 Structured Streaming for the Spark integration lab.

M14-L01 - Batch vs Streaming

Why this matters
Engineers must choose streaming because latency requirements justify it, not because it sounds more advanced.

Start with the simple idea
Batch waits for a bounded collection such as a daily file. Streaming treats new records as an ongoing sequence and processes them as they arrive or in small increments. Both can be correct.

Technical model
Latency is the central requirement: “within a day” may fit batch; “within seconds” may require streaming.

Streaming adds operational state: source position, event identity, ordering, checkpoints, lag and long-running failure recovery.

A streaming system may still process in micro-batches internally. Spark Structured Streaming does this by default.

Code / command pattern
```
mode = 'stream' if required_latency_seconds <= 30 else 'batch'
```

Worked reasoning example
A finance dashboard refreshed each morning does not need Kafka just because Kafka exists. Fraud alerts that must react within seconds may justify a stream. The choice comes from SLA, volume, failure tolerance and consumer behavior.

Common mistakes and debugging
- Calling streaming real-time without defining latency.
- Using streaming for tiny daily files with no latency requirement.
- Ignoring the operational cost of always-running services.

Engineering decision
Choose the simplest architecture that meets freshness/latency/reliability requirements.

Before practice

In G01, compare the same five records processed as a bounded batch and as individual arriving events. State what changes operationally.

Explain it without notes

Explain the concept in 3-5 sentences, name the Kafka evidence you would inspect (topic/partition/offset/group/checkpoint/lag), and state one failure mode that could lose, duplicate, reorder, or indefinitely delay business events.

M14-L02 - Events, Brokers and Topics

Why this matters

Kafka vocabulary becomes easy once every object has a job.

Start with the simple idea

An event is a fact that happened. A producer sends events. Kafka brokers persist them in named topics. Consumers read them. Kafka is not just a queue where a message must disappear after one reader sees it.

Technical model

A topic is a logical event stream. Topics are split into partitions for scale and ordering domains.

Events normally include key, value, timestamp and Kafka metadata such as topic/partition/offset.

Retention means an event can remain available after consumption, enabling another consumer or replay subject to retention policy.

Code / command pattern

```
event = {'event_id':'E1','order_id':'O7','status':'PAID'}
```

Worked reasoning example

An order service emits ORDER_PAID. Analytics, notification and fraud consumers may each read that event independently using different consumer groups. The event is a business fact, not a command to mutate arbitrary shared state.

Common mistakes and debugging

- Using an event with no stable identity.
- Putting unrelated business domains into one topic without ownership/schema reasoning.
- Assuming consumption deletes the Kafka record.

Engineering decision

Define event meaning and ownership before topic naming.

Before practice

For one supplied order event, write its business meaning, stable event ID, key, value fields and likely consumers.

Explain it without notes

Explain the concept in 3-5 sentences, name the Kafka evidence you would inspect (topic/partition/offset/group/checkpoint/lag), and state one failure mode that could lose, duplicate, reorder, or indefinitely delay business events.

M14-L03 - Producers and Message Keys

Why this matters

The key influences partitioning and therefore ordering/scaling behavior.

Start with the simple idea

A producer serializes a business event and sends it to a topic. A key such as order_id lets related events consistently target the same partition under a stable partitioning strategy.

Technical model

Producer delivery is asynchronous in common clients; delivery reports/flush matter for evidence.

The message key should represent the business entity whose relative order matters.

Keys do not magically give global ordering across all partitions.

Code / command pattern

```
producer.produce('m14-events', key=order_id, value=json.dumps(event), callback=delivery_report)
producer.flush()
```

Worked reasoning example

If O42 emits CREATED then PAID then SHIPPED, keying by O42 keeps those related records in the same partition under the same partitioning strategy. Keying randomly can destroy that entity-level order.

Common mistakes and debugging

- Using no key when per-entity order matters.
- Using a high-level category key that creates one hot partition.
- Exiting without flushing/handling delivery errors.

Engineering decision

Choose keys from ordering and distribution requirements, then measure skew.

Before practice

G03 publishes keyed events. Record topic, partition and delivery result for at least five events.

Explain it without notes

Explain the concept in 3-5 sentences, name the Kafka evidence you would inspect (topic/partition/offset/group/checkpoint/lag), and state one failure mode that could lose, duplicate, reorder, or indefinitely delay business events.

M14-L04 - Consumers and Safe Processing

Why this matters
The dangerous question is not “did I read it?” but “when is it safe to mark progress?”

Start with the simple idea
A consumer reads records from assigned partitions. Your business processing may write to a database/file/API. Only after successful handling should progress be considered safe under your chosen delivery model.

Technical model
Consumers poll records and receive topic/partition/offset metadata.

Automatic commits can advance group position independently of whether downstream business work completed.

Manual/controlled commits make the success boundary explicit, but sink idempotency still matters if a crash happens after the sink write but before the commit.

Code / command pattern
```
msg = consumer.poll(1.0)
process(msg)
consumer.commit(message=msg, asynchronous=False)
```

Worked reasoning example
Suppose an event writes a payment row successfully but the process crashes before committing the Kafka offset. On restart the event may be delivered again. A unique event_id/upsert makes the sink replay-safe.

Common mistakes and debugging
- Committing before business processing.
- Assuming manual commit means exactly once everywhere.
- Swallowing processing errors and still advancing offsets.

Engineering decision
Design the source progress boundary together with sink idempotency.

Before practice
G04 processes records and commits only after the processing function succeeds. Save offset evidence.

Explain it without notes
Explain the concept in 3-5 sentences, name the Kafka evidence you would inspect (topic/partition/offset/group/checkpoint/lag), and state one failure mode that could lose, duplicate, reorder, or indefinitely delay business events.

M14-L05 - Partitions and Ordering

Why this matters
Partitions provide parallelism but define the practical ordering boundary.

Start with the simple idea

A topic can have several partitions. Records inside one partition have an ordered offset sequence. Different partitions progress independently; Kafka does not promise one global order across them.

Technical model

Partition count influences maximum parallel consumption in a consumer group.

Same-key routing helps preserve entity order, but changing key/partition strategy can affect that assumption.

Too few partitions limit parallelism; too many add metadata/connection/operational overhead.

Code / command pattern

```
# conceptual metadata
(topic='orders', partition=1, offset=104)
```

Worked reasoning example

A topic has 3 partitions. O1 events are in partition 0 and O2 events in partition 2. You may know O1:CREATED precedes O1:PAID from offsets in partition 0, but you cannot infer whether O1:PAID globally happened before O2:CANCELLED merely from their partition offsets.

Common mistakes and debugging

- Saying Kafka gives global ordering.
- Increasing partitions without checking key/order implications.
- Using one key for nearly all traffic and creating a hot partition.

Engineering decision

Partition based on throughput plus the smallest ordering domain the business actually needs.

Before practice

G02 groups supplied events by partition and proves order only within each partition.

Explain it without notes

Explain the concept in 3-5 sentences, name the Kafka evidence you would inspect (topic/partition/offset/group/checkpoint/lag), and state one failure mode that could lose, duplicate, reorder, or indefinitely delay business events.

M14-L06 - Offsets, Group Position and Replay

Why this matters

Offsets are the coordinates of records inside a partition and the basis for controlled replay.

Start with the simple idea

Each partition has offsets 0,1,2,... Consumers read records at positions; a consumer group tracks its progress. Replaying means deliberately reading older offsets again, normally because downstream state needs repair or a new consumer starts.

Technical model

An offset is partition-scoped: offset 10 in partition 0 is unrelated to offset 10 in partition 1.

Committed group offsets are progress metadata, not proof that downstream data is correct.

Replay must be designed with idempotent/replace-safe downstream behavior.

Code / command pattern

```
consumer.assign([TopicPartition('m14-events', 0, start_offset)])
```

Worked reasoning example

If a downstream defect affected events from offset 120-150, a controlled replay can start at 120. Without event IDs/upserts, replay may duplicate business rows. Therefore “can Kafka replay?” and “can my pipeline safely replay?” are separate questions.

Common mistakes and debugging

- Resetting offsets in production without a scoped recovery plan.
- Treating committed offsets as data-quality evidence.
- Replaying into append-only sinks that cannot deduplicate.

Engineering decision

Backfill/replay procedures must name the affected partitions/offsets, sink behavior and validation checks.

Before practice

G05 seeks to an explicit offset in a disposable exercise group and records the replayed offsets.

Explain it without notes

Explain the concept in 3-5 sentences, name the Kafka evidence you would inspect (topic/partition/offset/group/checkpoint/lag), and state one failure mode that could lose, duplicate, reorder, or indefinitely delay business events.

M14-L07 - Consumer Groups and Scaling

Why this matters

Groups let multiple workers share one logical subscription.

Start with the simple idea

Consumers with the same group.id cooperate: partitions are assigned across group members so one partition is processed by one consumer in the group at a time. Different groups can independently read the same topic.

Technical model

A group cannot actively process one partition with two members at the same moment; extra consumers beyond partition count can sit idle.

Membership changes trigger partition reassignment/rebalancing behavior.

Separate applications use separate groups when they need independent progress.

Code / command pattern

```
conf = {'bootstrap.servers':'localhost:9092','group.id':'m14-orders','enable.auto.commit':False}
```

Worked reasoning example

A 3-partition topic with a group of 2 consumers can split 2+1 partitions. Adding a fourth consumer does not create more source parallelism unless partition count changes. A fraud app needs a different group from analytics so it has independent offsets.

Common mistakes and debugging

- Assuming consumers in the same group all receive every record.
- Adding workers without enough partitions.
- Sharing a group ID across unrelated applications.

Engineering decision

Scale groups based on partition count, processing time and lag, not worker count alone.

Before practice

Run G06 in two terminals with the same group ID and record partition assignment changes.

Explain it without notes

Explain the concept in 3-5 sentences, name the Kafka evidence you would inspect (topic/partition/offset/group/checkpoint/lag), and state one failure mode that could lose, duplicate, reorder, or indefinitely delay business events.

M14-L08 - Delivery Semantics and Idempotency

Why this matters

At-most-once, at-least-once and exactly-once are end-to-end properties, not marketing labels on one component.

Start with the simple idea

At-most-once may lose work but avoids redelivery. At-least-once can redeliver after failure, so sinks must tolerate duplicates. Exactly-once requires coordinated semantics across source, processing and sink boundaries.

Technical model

Kafka can provide strong producer/transaction features, but a custom consumer writing to an external database can still duplicate if failure happens between sink write and offset commit.

Stable event_id plus uniqueness/upsert is a common practical defense under at-least-once delivery.

Never claim exactly once without defining the full path and failure boundary.

Code / command pattern

```
if event_id not in processed_ids:
    apply_business_change(event)
    processed_ids.add(event_id)
```

Worked reasoning example

Process E9, write the sink, crash before commit, then receive E9 again. With a unique event_id and deterministic upsert, the second attempt becomes harmless/reconcilable. Without it, account balances or counts can double.

Common mistakes and debugging

- Calling enable.idempotence on a producer end-to-end exactly once.
- Deduping by mutable payload instead of stable event identity.
- Ignoring side effects such as emails/API calls.

Engineering decision

State the guarantee per boundary and design safe replay.

Before practice

G07 simulates a duplicate delivery and compares a naive sink with an event-id-aware sink.

Explain it without notes

Explain the concept in 3-5 sentences, name the Kafka evidence you would inspect (topic/partition/offset/group/checkpoint/lag), and state one failure mode that could lose, duplicate, reorder, or indefinitely delay business events.

M14-L09 - Serialization, Schemas and Evolution

Why this matters

Consumers need stable meaning, not just bytes that happen to parse.

Start with the simple idea

Kafka values are bytes. Applications serialize business events, often as JSON, Avro or Protobuf. A schema defines fields/types/meaning and how change is governed.

Technical model

JSON is beginner-friendly but does not automatically enforce compatibility.

Schema evolution must consider required vs optional fields, renamed meaning, types and old/new consumer coexistence.

A malformed/unknown event should have an explicit reject/quarantine/dead-letter policy rather than disappearing silently.

Code / command pattern

```
required={'event_id','order_id','event_type','event_time'}
missing = required - set(event)
```

Worked reasoning example

Adding optional `currency` may be compatible if old consumers ignore unknown fields and new logic handles absence. Renaming `order_id` to `id` without a compatibility plan can break every consumer. Type changes can be semantically breaking even if JSON parses.

Common mistakes and debugging

- Equating parseable JSON with valid business schema.
- Silently defaulting missing business keys.
- Changing field meaning without version/contract communication.

Engineering decision
Treat event schemas as producer-consumer contracts with owners and compatibility rules.

Before practice
G08 validates good/bad JSON events and writes reject reasons.

Explain it without notes
Explain the concept in 3-5 sentences, name the Kafka evidence you would inspect (topic/partition/offset/group/checkpoint/lag), and state one failure mode that could lose, duplicate, reorder, or indefinitely delay business events.

M14-L10 - Python Producer and Consumer Workflow

Why this matters
A real hands-on workflow connects topic creation, publishing, consuming, error handling and evidence.

Start with the simple idea
The producer emits keyed JSON events. The consumer uses a named group, validates/deserializes, processes successfully, then commits progress. Logs include partition and offset so failures are traceable.

Technical model
Use delivery callbacks/flush to prove producer delivery.

Use unique group IDs for isolated exercises; stable service group IDs for real applications.

Close/flush clients cleanly and keep credentials/config outside code when security is introduced.

Code / command pattern
```
python M14_G03_PRODUCER.py
python M14_G04_CONSUMER.py
```

Worked reasoning example
Run producer for six order events, then a consumer. Capture partition/offset. Deliberately reject one malformed event without committing it as successfully processed unless your error policy explicitly moves it to a durable reject path.

Common mistakes and debugging
- Treating a printed “sent” line as broker acknowledgement.
- Hardcoding credentials in future secured clusters.
- Running infinite consumer loops with no shutdown/error plan.

Engineering decision
Operational evidence is part of the lab: delivery reports, consumed offsets, failures and clean shutdown.

Before practice
G03-G06 form the local Kafka workflow. Do not mark them complete from static code reading.

Explain it without notes

Explain the concept in 3-5 sentences, name the Kafka evidence you would inspect (topic/partition/offset/group/checkpoint/lag), and state one failure mode that could lose, duplicate, reorder, or indefinitely delay business events.

M14-L11 - Spark Structured Streaming + Kafka

Why this matters

Structured Streaming lets you reuse DataFrame reasoning on an unbounded source while Spark tracks progress in checkpoints.

Start with the simple idea

Spark treats the stream like a continuously growing table. `readStream` creates a streaming DataFrame; transformations look similar to batch DataFrames; `writeStream` starts a long-running query.

Technical model

Kafka source values/keys arrive as bytes and need explicit casting/parsing.

CheckpointLocation stores query progress/state needed for restart. Do not casually delete it to make errors disappear.

Default micro-batch processing executes small batches repeatedly; end-to-end guarantees also depend on the sink and query design.

Code / command pattern

```
raw=(spark.readStream.format('kafka')
 .option('kafka.bootstrap.servers','localhost:9092')
 .option('subscribe','m14-events').load())
parsed=raw.selectExpr('CAST(key AS STRING) key','CAST(value AS STRING) value','partition','offset')
```

Worked reasoning example

A streaming query parses orders, validates event fields and writes to a replay-safe sink with a stable checkpoint. On restart Spark uses checkpoint/source progress instead of starting from an arbitrary position. If the sink itself is non-idempotent, checkpointing alone does not make external side effects exactly once.

Common mistakes and debugging

- Deleting checkpoints whenever a query fails.
- Ignoring event time and late data for stateful aggregations.
- Forgetting to install the Spark Kafka connector package.

Engineering decision

Checkpoint paths are durable operational state. Version query/state changes deliberately.

Before practice

G10 runs a Kafka source through Structured Streaming, prints selected metadata and uses a checkpoint location.

Explain it without notes

Explain the concept in 3-5 sentences, name the Kafka evidence you would inspect (topic/partition/offset/group/checkpoint/lag), and state one failure mode that could lose, duplicate, reorder, or indefinitely delay business events.

M14-L12 - Lag, Failures and Streaming Operations

Why this matters

A streaming job can be “running” while falling minutes or hours behind.

Start with the simple idea

Lag compares how far a consumer group is behind the end of a partition. Operational health also includes input rate, processing rate, error rate, poison records, rebalance behavior and sink latency.

Technical model

Lag is partition/group specific. High lag can come from slow processing, hot partitions, downstream failures, insufficient consumers or traffic spikes.

Poison events need explicit handling so one bad record does not block progress forever or vanish silently.

Recovery runbooks should define replay boundaries, checkpoint/group behavior and reconciliation after restart.

Code / command pattern

```
lag = max(0, end_offset - current_group_offset)
```

Worked reasoning example

Partition 0 end=1000/current=995 has lag 5; partition 1 end=900/current=300 has lag 600. Adding a worker helps only if group/partition assignment and downstream capacity allow it. First inspect which partition and why it is slow.

Common mistakes and debugging

- Looking only at total lag and missing one hot partition.
- Resetting group offsets to latest to hide backlog.
- Retrying poison events forever without a durable error policy.

Engineering decision

Respond from evidence: partition lag -> processing time/errors -> downstream -> key skew -> capacity -> recovery validation.

Before practice

G09 calculates lag from sample offsets. Integration workshop introduces a malformed record and a consumer restart/replay scenario.

Explain it without notes

Explain the concept in 3-5 sentences, name the Kafka evidence you would inspect (topic/partition/offset/group/checkpoint/lag), and state one failure mode that could lose, duplicate, reorder, or indefinitely delay business events.